

Machine Learning Algorithms

A Beginner's Learning Path

Study these in order, one phase at a time.

35 algorithms across 9 phases — from your first line of code to deep learning.

How to Use This Guide

This is a roadmap, not a race. The algorithms are arranged so that each one builds on what came before — follow the order and the harder topics will feel like natural next steps rather than sudden walls.

For each algorithm, aim to: (1) understand the **intuition** — what problem it solves and why; (2) learn the **core idea or math** at a high level; and (3) **implement it**, first with scikit-learn, then ideally once from scratch to cement the concept. Always test on a small real dataset so the idea sticks.

A Few Tips Before You Start

- **Go slow on Phases 0–2.** A solid grasp of linear and logistic regression makes everything after much easier.
- **Code along.** Reading about an algorithm is not the same as using it. Build something small each time.
- **Use scikit-learn** for classical ML (Phases 1–6) and **PyTorch or TensorFlow/Keras** for deep learning (Phase 7).
- **It's fine to skip Phase 6** on a first pass and return when a project needs it.
- **Don't memorize — understand.** Knowing *when* and *why* to use an algorithm matters more than reciting its formula.

Phase 0 — Foundations (Before the Algorithms)

Don't skip this. Every algorithm below rests on these basics. You don't need to master them first — just enough to follow along, then deepen as you go.

0.1 Python + NumPy / pandas

TOOLS

The language and libraries you'll write everything in. Learn arrays, dataframes, and basic plotting with matplotlib.

0.2 Math essentials

MATH

Linear algebra (vectors, matrices), basic calculus (derivatives, gradients), and probability & statistics (mean, variance, distributions).

0.3 ML concepts

THEORY

Training vs. test data, overfitting vs. underfitting, the bias–variance tradeoff, and evaluation metrics (accuracy, precision, recall, MSE).

Phase 1 — Supervised Learning — Regression

Start here. Regression predicts a number (a price, a temperature). Linear regression is the simplest possible model, so it's the perfect first algorithm to truly understand.

1 Linear Regression

REGRESSION

Fits a straight line to data. The foundation of nearly everything in ML — learn the cost function and gradient descent here.

2 Polynomial Regression

REGRESSION

Extends linear regression to curves. Your first lesson in model complexity and overfitting.

3 Ridge Regression (L2)

REGRESSION

Adds a penalty to shrink coefficients and reduce overfitting. Introduces regularization.

4 Lasso Regression (L1)

REGRESSION

Like Ridge, but can shrink coefficients to zero — doing automatic feature selection.

5 Elastic Net

REGRESSION

Combines Ridge and Lasso. Good to know once you understand both.

Phase 2 — Supervised Learning — Classification

Now predict categories (spam / not-spam, cat / dog). These are the workhorse classifiers. Logistic regression connects directly to what you just learned.

6 Logistic Regression

CLASSIFICATION

Despite the name, it's a classifier. The natural next step after linear regression.

7	K-Nearest Neighbors (KNN) CLASSIFICATION	Classifies a point by looking at its closest neighbors. Intuitive, no training phase — great for building intuition.
8	Naive Bayes CLASSIFICATION	A probabilistic classifier based on Bayes' theorem. Fast, simple, and surprisingly effective for text.
9	Decision Trees CLASSIFICATION	Splits data with yes/no questions. Highly interpretable and the building block for ensembles in Phase 3.
10	Support Vector Machines (SVM) CLASSIFICATION	Finds the optimal boundary between classes. Introduces the powerful 'kernel trick' for non-linear data.

Phase 3 — Ensemble Methods

The idea: combine many weak models into one strong one. These often win real-world competitions. Learn them right after decision trees, since most are built on trees.

11	Bagging ENSEMBLE	Trains many models on random subsets of data and averages them. The concept behind Random Forest.
12	Random Forest ENSEMBLE	Many decision trees voting together. A reliable, go-to algorithm for tabular data.
13	AdaBoost ENSEMBLE	Your first boosting method — models learn sequentially from previous mistakes.
14	Gradient Boosting ENSEMBLE	Builds models that correct the errors of prior ones using gradients. Powerful and widely used.
15	XGBoost / LightGBM / CatBoost ENSEMBLE	Fast, optimized gradient boosting libraries. The top choice for structured/tabular data today.

Phase 4 — Unsupervised Learning — Clustering

No labels here — the algorithm finds structure on its own by grouping similar points. A different mindset from Phases 1–3, so take your time.

16	K-Means Clustering CLUSTERING	Groups data into K clusters. The classic, intuitive starting point for unsupervised learning.
----	---	---

17	Hierarchical Clustering CLUSTERING	Builds a tree of nested clusters — no need to choose K in advance.
18	DBSCAN CLUSTERING	Density-based clustering that finds arbitrary shapes and detects outliers.
19	Gaussian Mixture Models (GMM) CLUSTERING	A probabilistic 'soft' clustering where points can belong to multiple clusters.

Phase 5 — Dimensionality Reduction

Real data has many features. These techniques compress them while keeping the important signal — useful for visualization and speeding up other algorithms.

20	Principal Component Analysis (PCA) DIM. REDUCTION	The foundational technique — finds the directions of greatest variance in your data.
21	Linear Discriminant Analysis (LDA) DIM. REDUCTION	Reduces dimensions while keeping classes separable. (Different from the topic model with the same initials.)
22	t-SNE DIM. REDUCTION	Excellent for visualizing high-dimensional data in 2D or 3D.
23	UMAP DIM. REDUCTION	A faster, often clearer modern alternative to t-SNE.

Phase 6 — Special Topics (Optional Detour)

Useful, focused tools you can pick up when a project needs them. Not strictly sequential — dip in as relevant.

24	Apriori / FP-Growth ASSOCIATION	Association rule mining — the 'customers who bought X also bought Y' algorithms.
25	Anomaly Detection (Isolation Forest, One-Class SVM) ANOMALY	Spotting rare, unusual data points — used in fraud and fault detection.

Phase 7 — Neural Networks & Deep Learning

The frontier. Start only after you're comfortable with the basics above — the math (gradients, the chain rule) pays off here. Learn the perceptron first, then build up.

26	Perceptron NEURAL NET	A single artificial neuron — the smallest unit of a neural network.
27	Multilayer Perceptron (MLP) NEURAL NET	Stacked neurons trained with backpropagation. The core deep learning concept.
28	Convolutional Neural Networks (CNN) DEEP LEARNING	Specialized for images and spatial data — powers most computer vision.
29	RNN / LSTM / GRU DEEP LEARNING	Built for sequences like text and time series, with memory of past inputs.
30	Autoencoders DEEP LEARNING	Learn compressed representations — used for denoising and anomaly detection.
31	Generative Adversarial Networks (GANs) DEEP LEARNING	Two networks compete to generate realistic new data (images, audio).
32	Transformers DEEP LEARNING	The architecture behind modern LLMs and state-of-the-art NLP. Learn attention here.

Phase 8 — Reinforcement Learning

An agent learns by trial and error through rewards. A distinct paradigm — best saved for last once everything else feels solid.

33	Q-Learning REINFORCEMENT	The classic value-based method — learns the value of actions in states.
34	Deep Q-Networks (DQN) REINFORCEMENT	Combines Q-learning with neural networks to handle complex environments.

35 Policy Gradient Methods

REINFORCEMENT

Directly learn the policy. Includes REINFORCE, A2C, and PPO.

What Comes After

Once you've worked through these, you'll have a strong, well-rounded foundation. Natural next steps include: diving deeper into a specialization (computer vision, NLP, or reinforcement learning), learning to deploy models (MLOps), studying model evaluation and tuning in depth (cross-validation, hyperparameter search), and reading original papers for the algorithms that interest you most.

Most importantly, build projects. A handful of end-to-end projects — even small ones — will teach you more than any single algorithm in isolation.

Take it one phase at a time. You've got this.